

Software Design Specification

VisionEdge: Latency-Optimized Local Perception & TTS Pipeline

Revision 1.0

Prepared by:

Ayan Gattani (23BCT0059) | Madhav Juneja (23BCT0111) | Adheesh Garg (23BCT0122)

Instructor: Dr. Swarnalatha P

Course: Software Engineering Lab | Lab Section: L23+L24

Date: 18 February 2026

Revision History

Version	Name	Reason For Changes	Date
1.0	Madhav Juneja, Ayan Gattani, Adheesh Garg	Initial Draft of Software Design Specification	February 2026

Approved By

Approvals should be obtained from the course instructor and all team members.

Name	Signature	Department / Role	Date
Dr. Swarnalatha P		Course Instructor, Software Engineering	

Table of Contents

1. Introduction this is about our project, read and understand it well	
5	
1.1 Purpose	5
1.2 System Overview.....	5
1.3 Design Map	5
1.4 Supporting Materials.....	5
1.5 Definitions and Acronyms	6
2. Design Considerations	7
2.1 Assumptions.....	7
2.2 Constraints	7
2.3 System Environment	7
2.4 Design Methodology	7
2.5 Risks and Volatile Areas.....	8
3. Architecture.....	9
3.1 Overview	9
3.2 Subsystem 1 — Vision Perception Module (VPM)	9
3.3 Subsystem 2 — Text Generation Engine (TGE)	9
3.4 Subsystem 3 — TTS Pipeline Module (TPM).....	10
3.5 Strategy — Latency Optimization.....	10
4. Database Schema.....	11
4.1 Tables, Fields, and Relationships	11
4.1.1 Databases.....	11
4.1.2 New Tables.....	11
4.1.3 New Fields	11
4.1.4 Field Changes.....	12
4.1.5 All Other Changes.....	12
4.2 Data Migration	12
5. High Level Design.....	13
5.1 Camera Input & Frame Capture Component	13
5.2 Vision Perception Component	13
5.3 Text Generation Component.....	13
5.4 TTS Pipeline Component.....	13
5.5 Application Integration & UI Component	14
6. Low Level Design.....	15

6.1 Module: CameraManager	15
6.2 Module: VisionInferenceEngine	15
6.3 Module: TextGenerationEngine	15
6.4 Module: TTSEngine	16
6.5 Module: AccessibilityUIController	16
7. User Interface Design	17
7.1 Application Controls.....	17
7.2 Screen 1 — Home / Main Screen	17
7.3 Screen 2 — Settings Screen.....	17
7.4 Screen 3 — Error / Alert States	18
Appendix A: Project Timeline	19

1. Introduction

1.1 Purpose

This Software Design Specification (SDS) describes the architectural and detailed design of VisionEdge, an offline, mobile-based assistive application that converts a smartphone into a real-time visual aid for visually impaired users. The system performs on-device visual perception and generates immediate audio feedback using a lightweight neural Text-to-Speech (TTS) pipeline.

This document is intended to serve as a comprehensive blueprint for implementation, guiding the development team through architectural decisions, module decomposition, interface definitions, and data design. It translates the requirements stated in the Software Requirements Specification (SRS) into actionable design constructs.

1.2 System Overview

VisionEdge operates entirely offline on a smartphone, capturing live video frames from the device camera and processing them through a compressed, on-device computer vision model. Detected objects and scene elements are converted into natural language descriptions, which are then passed to a neural TTS module to generate spoken audio feedback to the user.

The key innovation of VisionEdge lies in its end-to-end latency optimization — achieving perception-to-speech feedback within 2 seconds — while maintaining complete privacy by avoiding cloud services entirely. The system is organized into three primary subsystems: the Vision Perception Module, the Text-to-Speech Module, and the Application Integration Layer.

1.3 Design Map

This document is organized as follows:

- Section 2 — Design Considerations: outlines assumptions, constraints, system environment, design methodology, and risks.
- Section 3 — Architecture: provides the top-level structural decomposition and describes each major subsystem.
- Section 4 — Database Schema: covers local data storage, model metadata, and any persistent configuration.
- Section 5 — High Level Design: describes the major components, their responsibilities, and interactions.
- Section 6 — Low Level Design: provides detailed module-level descriptions supporting direct implementation.
- Section 7 — User Interface Design: describes accessibility-focused screen designs and interaction flows.

1.4 Supporting Materials

The following related project artifacts complement this SDS:

- Software Requirements Specification (SRS) — VisionEdge v1.0, January 2026
- Software Engineering Lab Submission 1 — Project Overview, SDLC Model, Gantt Chart, PERT/WBS

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications
- Technical documentation for ONNX Runtime, TFLite, and mobile ML inference frameworks

1.5 Definitions and Acronyms

Term / Acronym	Definition
TTS	Text-to-Speech — conversion of text to synthesized spoken audio
On-Device AI	Execution of AI inference locally on the mobile device, without cloud dependency
Edge Inference	Running machine learning models on edge devices (smartphones, embedded systems)
ONNX	Open Neural Network Exchange — an interoperable model format for ML frameworks
TFLite	TensorFlow Lite — a lightweight ML inference engine for mobile devices
Quantization	Model compression technique that reduces precision of weights to lower memory and latency
Knowledge Distillation	Technique of training a smaller student model using outputs of a larger teacher model
NPU	Neural Processing Unit — specialized hardware for AI inference on mobile SoCs
Latency	Time delay between input capture and final audio output generation
SRS	Software Requirements Specification
WBS	Work Breakdown Structure

2. Design Considerations

2.1 Assumptions

- The target smartphone device is equipped with a functional rear camera capable of capturing video at a minimum of 15 fps.
- The device supports one of the following mobile ML inference frameworks: TensorFlow Lite, ONNX Runtime Mobile, or equivalent.
- Adequate ambient lighting is available for effective visual perception in typical indoor and outdoor environments.
- The mobile operating system provides stable and accessible APIs for camera input, audio output, and thread management.
- The team has access to appropriate pre-trained teacher models for knowledge distillation (e.g., YOLOv8, Coqui TTS).
- Metrics and benchmarking will be deferred to the testing phase; performance thresholds are defined in the SRS.

2.2 Constraints

- The system must operate fully offline with zero dependence on internet connectivity or cloud-based APIs.
- End-to-end latency must not exceed 2 seconds from frame capture to audio playback, as required by PERF-1 in the SRS.
- All ML models must be compressed (quantized and/or pruned) to fit within typical mobile memory budgets (under 200 MB combined).
- The application must not persistently store raw video frames or audio captures, per SEC-2 of the SRS.
- Development must adhere to the iterative incremental SDLC model and the timeline defined in Submission 1.
- The project is developed using standard open-source tools and frameworks only.

2.3 System Environment

VisionEdge targets modern Android smartphones (API level 26+) running on ARMv8-A processors with at least 3 GB RAM and 64 GB storage. The system leverages available on-device accelerators — CPU, GPU, or Neural Processing Units (NPUs) — depending on device availability. The application is developed using a cross-platform mobile framework with native ML inference bindings.

Processing occurs entirely on-device. The app shares device resources with other running applications and must coexist gracefully without significantly degrading the overall system performance or battery life.

2.4 Design Methodology

VisionEdge follows a modular, pipeline-oriented design approach. The system is decomposed into three independently testable subsystems — Vision, TTS, and Application Layer — that are developed in parallel (Phases 4 and 5 of the project timeline overlap) and integrated in Phase 6.

Each module is designed to be self-contained with clearly defined input/output contracts. This allows independent unit testing and facilitates model swapping during the optimization phase.

The Iterative Incremental SDLC model governs overall development, with each increment delivering a functional, demonstrable subsystem capability.

2.5 Risks and Volatile Areas

Risk	Likelihood	Mitigation Strategy
On-device latency exceeds 2s threshold on low-end devices	High	Apply INT8 quantization; benchmark early on target device; fall back to simpler model
TTS voice naturalness insufficient for usability	Medium	Evaluate multiple distilled TTS models; allow model-swap without app rebuild
Camera API differences across Android versions	Medium	Use Camera2 API with Camera X compatibility wrapper; test on multiple OS versions
Model memory footprint causes OOM on constrained devices	Medium	Implement model unloading when app is backgrounded; set max memory budget per model
Poor detection accuracy in low-light or cluttered scenes	Low	Set confidence threshold; surface 'low confidence' audio cue to user

3. Architecture

VisionEdge is structured as a three-tier, pipeline-driven mobile architecture. The core pipeline flows from Camera Input → Vision Perception Module → Text Generation → TTS Pipeline → Audio Output. The Application Layer orchestrates this pipeline, manages device resources, and presents an accessible user interface.

3.1 Overview

The system architecture decomposes into the following top-level subsystems:

- Vision Perception Module (VPM): Receives raw video frames, runs on-device object detection and scene classification, and outputs a structured list of detected entities with confidence scores.
- Text Generation Engine (TGE): Converts structured detection results into concise, natural-language descriptions suitable for audio narration.
- TTS Pipeline Module (TPM): Accepts text descriptions and synthesizes spoken audio using an offline neural TTS model, outputting an audio stream to the device speaker.
- Application Integration Layer (AIL): Acts as the pipeline coordinator — initializing camera capture, scheduling frame processing, threading TTS calls, and managing the accessibility-focused UI.

The critical path runs through the Vision Model and TTS Module in sequence. The TTS Module is treated as the critical path element given its dependency on completed detection output (as indicated in the PERT chart in Submission 1).

3.2 Subsystem 1 — Vision Perception Module (VPM)

The VPM handles all computer vision inference. It accepts raw camera frames (JPEG or YUV format), runs them through a compressed object detection model, and returns a structured detection payload.

Design decisions:

- Model format: TFLite (preferred for Android NPU acceleration) with ONNX as fallback.
- Model architecture: A distilled and quantized student model derived from YOLOv8-nano or MobileNet-SSD via knowledge distillation.
- Inference threading: Runs on a dedicated background thread to prevent UI blocking.
- Frame rate: Processes frames at adaptive intervals (target: 2–5 fps during active narration to reduce compute pressure).

3.3 Subsystem 2 — Text Generation Engine (TGE)

The TGE translates structured detection results (object labels + confidence scores + spatial positioning) into human-readable sentences. It applies rule-based templating supplemented by priority ranking (foreground objects weighted higher) to produce concise, non-redundant descriptions.

Design decisions:

- Rule-based templating is chosen over generative LLM-based captioning to ensure deterministic, low-latency text generation that fits within mobile constraints.

- Scene change detection logic prevents re-narrating static scenes, reducing audio output fatigue.

3.4 Subsystem 3 — TTS Pipeline Module (TPM)

The TPM converts text descriptions into speech audio using a lightweight offline neural TTS model. Candidate models include Coqui TTS (distilled) or VITS (compressed), both deployable on-device.

Design decisions:

- Model distillation reduces the full TTS model to a fraction of its original size while preserving intelligible voice output.
- Audio buffering ensures smooth playback even during inference delays.
- Audio output is routed through Android's AudioManager to support both speaker and headphone/Bluetooth output.

3.5 Strategy — Latency Optimization

To meet the sub-2-second latency requirement, the following strategies are adopted:

- Parallel pipeline execution: TTS synthesis begins as soon as the first sentence is formed, overlapping with any remaining detection processing.
- INT8 quantization applied to both vision and TTS models to reduce inference time by up to 3×.
- Frame-skip logic: Skips inference on frames that are visually similar to the previous frame (using frame difference threshold), reducing unnecessary computation.
- Thread pool management: Dedicated threads for camera capture, vision inference, text generation, and TTS synthesis to maximize CPU/GPU utilization.

4. Database Schema

VisionEdge does not use a traditional relational database for runtime operation. All core processing is in-memory and transient. However, a lightweight local storage layer (SQLite via Room on Android) is used for configuration persistence, model metadata, and user preference storage.

4.1 Tables, Fields, and Relationships

4.1.1 Databases

A single on-device SQLite database (visionedge.db) is used, managed through the Room persistence library. It stores application configuration and session metadata only. No raw video or audio data is persisted.

4.1.2 New Tables

Table Name	Related Tables	Description
UserPreferences	None	Stores user-configurable settings such as speech rate, feedback verbosity, and audio output preference
ModelMetadata	None	Stores metadata for currently loaded vision and TTS models (version, quantization type, file path)
SessionLog	None	Optional lightweight session log for developer diagnostics; not used in production builds

4.1.3 New Fields

Table Name	Field Name	Data Type	Allow Nulls	Field Description
UserPreferences	speechRate	FLOAT	No	TTS speech rate multiplier (0.5–2.0, default 1.0)
UserPreferences	verbosityLevel	INTEGER	No	Audio description verbosity: 1 = minimal, 2 = standard, 3 = detailed
UserPreferences	audioOutputMode	TEXT	No	Preferred audio output: SPEAKER, EARPIECE, or BLUETOOTH
ModelMetadata	modelType	TEXT	No	Model category: VISION or TTS
ModelMetadata	modelVersion	TEXT	No	Semantic version string of the deployed model (e.g., '1.2.0')
ModelMetadata	quantizationType	TEXT	Yes	Quantization applied: INT8, FLOAT16, or FLOAT32
SessionLog	avgLatencyMs	INTEGER	Yes	Average end-to-end latency for the session in milliseconds

4.1.4 Field Changes

No existing fields require modification. All tables in this design are newly introduced in Version 1.0.

4.1.5 All Other Changes

No stored procedures, indexes beyond primary keys, or external database dependencies are required. All queries are handled by the Room DAO (Data Access Object) layer.

4.2 Data Migration

Since VisionEdge is a new standalone application with no predecessor system, no data migration is required for the initial release. Future version upgrades that alter the schema will use Room's built-in migration framework to ensure backward compatibility.

5. High Level Design

This section describes the major functional components of VisionEdge, their responsibilities, and their interactions within the system pipeline.

5.1 Camera Input & Frame Capture Component

This component interfaces with the smartphone's Camera2 API (or CameraX wrapper) to initialize the camera session, configure capture parameters (resolution, focus mode, frame rate), and deliver frames to the Vision Perception Module.

Responsibilities:

- Open and configure the rear camera session on application startup.
- Continuously push video frames to the vision inference queue at an adaptive frame rate.
- Handle camera errors (unavailable, permission denied) and surface audio alerts via the TTS module.
- Release camera resources when the app is backgrounded or closed.

5.2 Vision Perception Component

Receives frames from the camera component and runs the compressed object detection model to produce structured detection output.

Responsibilities:

- Load and initialize the TFLite/ONNX vision model at startup.
- Pre-process input frames (resize to model input dimensions, normalize pixel values).
- Run inference and post-process output (apply non-maximum suppression, filter by confidence threshold).
- Output a DetectionResult object containing detected class labels, bounding boxes, and confidence scores.
- Implement frame-skip logic based on visual difference threshold to avoid redundant computation.

5.3 Text Generation Component

Converts DetectionResult objects into natural language scene descriptions.

Responsibilities:

- Apply priority ranking to detected objects (proximity-weighted, confidence-weighted).
- Generate sentence descriptions using rule-based templates (e.g., 'A person is detected ahead. A chair is to the left.').
- Apply scene-change detection — suppress re-narration if the scene has not changed significantly since the last cycle.
- Forward the generated text string to the TTS Pipeline Module.

5.4 TTS Pipeline Component

Synthesizes spoken audio from text descriptions and delivers it to the user.

Responsibilities:

- Load and initialize the offline neural TTS model at startup.

- Accept text strings and synthesize corresponding audio waveforms.
- Route audio output through the Android AudioManager to the appropriate audio device.
- Queue audio segments to prevent overlap when new descriptions arrive before prior audio completes.
- Handle audio output failures and surface error notifications.

5.5 Application Integration & UI Component

Orchestrates the full pipeline, manages application lifecycle, and presents an accessibility-focused interface to the user.

Responsibilities:

- Manage application state machine (IDLE, ACTIVE, PAUSED, ERROR).
- Initialize and coordinate all subsystem components at launch.
- Provide large-touch-target UI elements for start/stop control.
- Provide audio confirmations for all user actions (start, stop, errors).
- Manage foreground service to maintain pipeline operation when the display is off.

6. Low Level Design

This section provides implementation-level descriptions for each module, including class responsibilities, key methods, and data flow contracts.

6.1 Module: CameraManager

Wraps the Android CameraX library to provide a simplified frame-delivery interface.

Key methods:

- `startCapture()`: Binds camera use case and starts frame delivery to the inference queue.
- `stopCapture()`: Unbinds camera use case and releases resources.
- `onFrameAvailable(ImageProxy frame)`: Callback triggered per captured frame; dispatches frame to `VisionInferenceEngine`.
- `onCameraError(int error)`: Triggers TTS audio alert with error message.

6.2 Module: VisionInferenceEngine

Encapsulates TFLite model loading and inference execution.

Key methods:

- `loadModel(String modelPath)`: Loads the quantized TFLite model from app assets; configures GPU/NPU delegate if available.
- `runInference(Bitmap frame)`: Preprocesses frame, executes inference, post-processes output, returns `DetectionResult`.
- `computeFrameDifference(Bitmap prev, Bitmap curr)`: Returns mean pixel difference score; used for frame-skip decision.
- `isFrameSkippable(Bitmap frame)`: Returns true if frame difference is below threshold (scene unchanged).

`DetectionResult` data contract:

- `List<DetectedObject> objects` — each containing: String label, float confidence, `RectF` boundingBox.
- `long inferenceTimeMs` — inference duration for performance monitoring.

6.3 Module: TextGenerationEngine

Implements rule-based natural language generation from detection output.

Key methods:

- `generateDescription(DetectionResult result)`: Applies priority ranking and template rules to produce a narration string.
- `isSceneChanged(DetectionResult prev, DetectionResult curr)`: Compares object sets; returns true if significant change detected.
- `buildSentence(String label, String position)`: Constructs per-object sentence fragment based on spatial position heuristic.

Scene change logic: A new narration is triggered if (a) the set of top-K detected objects changes, or (b) confidence scores shift by more than a defined threshold, or (c) more than 5 seconds have elapsed since the last narration.

6.4 Module: TTSEngine

Wraps the offline neural TTS model inference and Android audio playback.

Key methods:

- `loadModel(String modelPath)`: Loads distilled TTS model from app assets.
- `synthesize(String text)`: Runs TTS inference and returns `AudioBuffer`.
- `playAudio(AudioBuffer buffer)`: Routes buffer to `AudioTrack` for playback via `AudioManager`.
- `enqueue(String text)`: Adds text to synthesis queue if prior audio is still playing.
- `onAudioError()`: Logs error; triggers fallback notification (vibration pattern).

6.5 Module: AccessibilityUIController

Manages the application UI and user interaction events.

Key methods:

- `onStartButtonPressed()`: Transitions state to `ACTIVE`; calls `CameraManager.startCapture()`; plays confirmation audio.
- `onStopButtonPressed()`: Transitions state to `IDLE`; calls `CameraManager.stopCapture()`; plays stop confirmation audio.
- `renderStateChange(AppState state)`: Updates UI to reflect current application state with large visual indicators.
- `announceAction(String message)`: Passes status messages to `TTSEngine` for immediate audio playback.

7. User Interface Design

7.1 Application Controls

The VisionEdge interface is designed with visual impairment accessibility as the first principle. All core functions are operable without any visual feedback from the screen. The following common behaviors apply across all screens:

- All interactive controls use large touch targets (minimum 72dp × 72dp) to accommodate imprecise touch input.
- Every user action triggers an immediate audio confirmation via the TTS engine.
- High-contrast color scheme (dark background, high-brightness text and icons) supports low-vision users.
- TalkBack compatibility: all controls are labeled with content descriptions for screen reader support.
- Haptic feedback (vibration) is used to supplement audio confirmation where appropriate.
- The app enters a full-screen mode during active assistance to minimize accidental touches.

7.2 Screen 1 — Home / Main Screen

The Home Screen is the primary interaction screen. It contains a single dominant Start/Stop toggle button occupying the center of the screen. The status of the system (IDLE or ACTIVE) is indicated by button color, an icon, and a spoken status announcement.

Screen elements:

- Title bar: 'VisionEdge' label in large font (40sp) at the top.
- Status indicator: Large text label ('Assistance OFF' / 'Assistance ON') below the title.
- Primary action button: Circular button occupying 60% of screen height; green when ACTIVE, grey when IDLE.
- Settings icon in the top-right corner for accessing user preferences.
- Battery and latency indicators (small, bottom of screen) for developer builds only.

7.3 Screen 2 — Settings Screen

The Settings Screen allows users to configure speech rate, verbosity level, and audio output preference. All settings changes are confirmed with an audio announcement of the new value.

Screen elements:

- Speech Rate slider: Ranges from 0.5× to 2.0× with step intervals; current value announced when adjusted.
- Verbosity selection: Three large radio buttons labeled 'Minimal', 'Standard', and 'Detailed'.
- Audio Output selection: Toggle between Speaker, Earpiece, and Bluetooth (Bluetooth only shown when connected device is available).
- Save and Back button: Confirms settings and returns to Home Screen with audio confirmation.

7.4 Screen 3 — Error / Alert States

When the system encounters critical errors (camera unavailable, model load failure, audio output failure), the app transitions to an Error State screen and immediately announces the error via TTS.

Behaviors:

- Large error icon and description displayed on screen.
- Audio announcement: 'VisionEdge has encountered an error. [Error description]. Please try again.'
- Single Retry button that attempts to reinitialize the affected subsystem.
- If retry fails, user is guided to restart the application via audio instructions.

Appendix A: Project Timeline

The following table summarizes the VisionEdge project phases, aligned with the Gantt Chart and PERT diagram presented in Software Engineering Lab Submission 1.

Phase	Weeks	Key Deliverables
Requirements, Analysis & Feasibility	1–2	SRS, Risk Assessment, User Needs Analysis
System Design (this document)	3–4	SDS, Architecture Diagrams, Technology Selection
Data & Model Preparation	5–7	Dataset collection, Teacher model setup, Distillation plan
Vision Model Development	8–11	Student model training, Quantization, TFLite conversion
TTS Pipeline Development (Parallel)	10–12	TTS model selection, Distillation, Audio latency optimization
App Development & Integration	13–15	Camera integration, Vision–TTS pipeline, Accessible UI
Testing & Optimization	16–17	Functional testing, Latency benchmarking, Bug fixing
Documentation & Submission	18	Final report, Demo video, Presentation

For detailed Gantt Chart and PERT Network Diagram, refer to Software Engineering Lab Submission 1.